Komputerowe systemy automatyki II

Ćw. 4 - sterowanie z wykorzystaniem komputera z systemem LINUX

Kajetan Powała-Mosiński i Karolina Grodź

4 listopada 2026

1 Cel ćwiczenia

Zadanie polegało na napisaniu programu w języku C dla minikomputera Raspberry Pi, który zintegruje się z edukacyjną płytką rozszerzeń MAB-94. Program miał cyklicznie odczytywać dane z potencjometru za pomocą przetwornika ADC, wyświetlać wynik na wyświetlaczu 7-segmentowym oraz linijce LED, a także sterować prędkością silnika krokowego proporcjonalnie do tej wartości (uwzględniając zmianę kierunku na podstawie przełącznika DIP).

2 Ogólny opis układu

Zestaw MAB-94 komunikuje się z Raspberry Pi bezpośrednio przez piny GPIO, obsługiwane w kodzie przez bibliotekę `wiringPi`. Sprzęt korzysta z wirtualnej, równoległej szyny danych. Dostęp do poszczególnych modułów (silnik, przetwornik ADC, wyświetlacz) realizowany jest poprzez wystawienie 3-bitowego adresu na liniach A0–A2 i sprzętowe zatwierdzenie operacji krótkimi impulsami sterującymi odczytem lub zapisem.

3 Opis najważniejszych elementów programu

Program został podzielony na funkcje odpowiedzialne za komunikację sprzętową oraz logikę sterowania. Najważniejsze z nich to:

- `outMAB()` **i** `inMAB()` – fundamentalne funkcje obsługujące szynę. Ustawiają odpowiedni adres modułu na płytce MAB, po czym wpisują lub odczytują 8-bitowe dane z przypisanych pinów GPIO.

```
void outMAB(int addr, int data) {
    setMABAddr(addr);
    writeMAB(data);
    delayMicroseconds(BUS_DELAY_US);
    digitalWrite (nWR, 0);
    delayMicroseconds(BUS_DELAY_US);
    digitalWrite (nWR, 1);
    delayMicroseconds(BUS_DELAY_US);
}
```

- `readADC()` – funkcja dokonująca pomiaru. Inicjuje konwersję na adresie przetwornika, a następnie w bezpiecznej pętli z limitem czasu czeka na flagę zakończenia przetwarzania. Surowy wynik jest potem uśredniany w pętli głównej.

```
int readADC(void) {
    unsigned int t0;
    outMAB(4, addr4 & ~ADCstart);
    delayMicroseconds(20);
    outMAB(4, addr4 | ADCstart);
    delayMicroseconds(20);

    t0 = micros();
    while ( ((inMAB(4) & IACstat) ? 1 : 0) != ADC_READY_LEVEL ) {
        if ((unsigned int)(micros() - t0) > ADC_TIMEOUT_US) break;
        delayMicroseconds(20);
    }
    return inMAB(2);
}
```

- `stepPeriod(adc)` – przelicza odczytaną wartość ADC na interwał czasowy w mikrosekundach. Funkcja ma zdefiniowaną strefę martwą – dla bardzo małych wartości ADC zwraca 0, wyłączając wysterowanie silnika.

```
unsigned int stepPeriod(int adc) {
    unsigned int sps;
    if (adc < ADC_DEADZONE) return 0;
    sps = SPS_MIN + (unsigned int)(SPS_MAX - SPS_MIN) *
          (unsigned int)(adc - ADC_DEADZONE) / (unsigned int)(255 -
    ADC_DEADZONE);
    return 1000000u / sps;
}
```

- **Asynchroniczna pętla główna** (`while(running)`) – program nie korzysta z funkcji blokujących typu `delay()`. Zamiast tego na bieżąco sprawdza aktualny czas systemowy funkcją `micros()` i wykonuje określone akcje (np. krok silnika) dopiero, gdy minie wymagana liczba mikrosekund.

```
while (running) {
    now = micros();
    if (period != 0 && (unsigned int)(now - lastStep) >= period) {
        lastStep += period;
        if ((unsigned int)(now - lastStep) >= period) lastStep = now;
        phase = cw ? (phase + 1) & 3 : (phase + 3) & 3;
        outMAB(0, OPh_seq[phase] | OKB_OFF);
    }
    // ... [odczyt ADC i aktualizacja diod] ...
    delayMicroseconds(200);
}
```

4 Wnioski

Oprogramowanie z powodzeniem zrealizowało wszystkie punkty zadania. Bezpośrednie sterowanie pinami GPIO pozwoliło na sprawną integrację z modułami płytki MAB-94. Zastosowanie nieblokującego odmierzania czasu oraz cyfrowe uśrednianie wyników z ADC okazały się kluczowe dla zachowania płynnej, bezproblemowej pracy silnika krokowego.